

Lecture 3: 随机算法——Maxcut

2024.3.5

Lecturer: 丁虎

Scribe: 沈俊杰

1 一些前置知识

下面介绍一些本篇讲义中需要用到的知识及定义。

Definition 1.1. 随机算法是一种在算法过程中引入随机函数，且随机函数的返回值直接或间接影响了算法的执行流程或执行结果。一般的随机算法分类有以下几种：

- Las Vegas 算法：算法总是返回正确的结果。（但不保证返回结果）
- Monte Carlo 算法：算法返回的结果未必正确，只能保证正确（错误）的概率。
- Sherwood 算法：消除输入样例对计算复杂度的影响。

Definition 1.2. 凸优化问题。

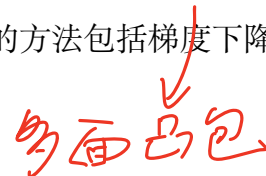
一般优化问题一般优化问题的形式为

$$\begin{aligned} \min \quad & f_0(x) \\ \text{s.t.} \quad & f_i(x) \leq 0, \quad i = 1, \dots, m \\ & h_i(x) = 0, \quad i = 1, \dots, n \end{aligned}$$

注意：凸优化问题要求：

- $f_0(x)$ 是下凸函数
- 可行域 (Feasible Domain)，即满足约束的 x 的范围，是一个凸域。

在机器学习等任务中，凸优化问题非常常见，一些经典的方法包括梯度下降被广泛应用。



半正定规划

Example 1.3. SDP problem:

变量: $\{x_{ij} | 1 \leq i, j \leq n\}$

目标: $\min \sum_{i,j} \alpha_{ij} x_{ij}$

约束: $\mathbb{X} = (x_{ij})$ 半正定

... (一些关于 x_{ij} 的线性不等式组)

$$\forall y \in \mathbb{R}^n$$

$$y^T \mathbb{X} y \geq 0$$

$\mathbb{R}^n$ 中的 Half space

Remark 1.4. SDP 问题是凸优化问题, 因为变量空间 $\mathbb{X}$ 是一个锥 (cone), 特殊的, 这类问题可以叫做 conic programming

无穷多半空间交集

Definition 1.5. 整数线性规划:

一类特殊的线性规划, 变量取值仅可为整数值。更为特殊的, 仅可为 0-1 值, 此时称为 0-1 规划。

数值优化 解决组合优化 (最小生成树...)

2 Max Cut

NP-hard

接下来我们考虑一个具体的问题——最大割问题。

Definition 2.1. Max Cut:

Input: $G = (V, E)$, $V = \{1, 2, \dots, n\}$ $\forall i, j \in V, w_{ij} \geq 0$

Output: $S = \arg \max_{S \subseteq V} w(S, V \setminus S)$, $w(S, V \setminus S) = \sum_{i \in S, j \notin S} w_{ij}$

max 割

有权无向图



图中跨两个割权重长度之和

我们使用 SDP 解决这个问题。首先转化一下问题:

$$\sum_{i \in S, j \notin S} w_{ij} = \frac{1}{2} \sum_{i \leq j} w_{ij} (1 - y_i y_j) \quad (1)$$

$$y_i = \begin{cases} +1, & i \in S \\ -1, & i \notin S \end{cases} \quad (2)$$

同侧 0

异侧 1

我们进一步做 Relaxation:

放松

$$y_i = \{\pm 1\}_n \Rightarrow v_i \in \mathbb{S}^n \quad (3)$$

n 维单位球

Remark 2.2. $\mathbb{S}^n$ 是 $\mathbb{R}^n$ 上的单位球面, 原先的 $y_i = \{\pm 1\}$ 是 $\mathbb{S}^1$, 后者是前者的特殊情况。注意, 这样的 Relaxation 会导致目标函数的最优值偏大 (可行域变大了)。我们记此时的 Problem 为 $\mathbb{B}$, 区别于原始的问题 $\mathbb{A}$

将 Relax 之后的形式带回 (1) 中, 我们令 $x_{ij} = \langle v_i, v_j \rangle$ 内积

$$\frac{1}{2} \sum_{i \leq j} w_{ij} (1 - y_i y_j) = \frac{1}{2} \sum_{i \leq j} w_{ij} (1 - x_{ij}) \quad (4)$$

可以看出, 此时的目标函数形式已经是 SPD 形式, 我们来证明 $\mathbb{X}$ 是半正定的。

Proof.

向量内积空间 $\rightarrow$

$$y^T \mathbb{X} y = \sum_{ij} y_i y_j x_{ij} = \sum_{ij} y_i y_j \langle v_i, v_j \rangle \quad (5)$$

$$= \sum_{ij} \langle y_i v_i, y_j v_j \rangle = \langle y_1 v_1 + y_2 v_2 + \cdots + y_n v_n, y_1 v_1 + y_2 v_2 + \cdots + y_n v_n \rangle \quad (6)$$

$$= \|y_1 v_1 + y_2 v_2 + \cdots + y_n v_n\|^2 \geq 0 \quad (7)$$

向量

□

Remark 2.3. 得到 $\mathbb{X}$, 如何得到最初始的 $\{v_1, \dots, v_n\}$?

事实上, $\mathbb{X}$ 和 $\{v_1, \dots, v_n\}$ 相差一个旋转因子, 不过这对问题的性质没有关系。对于正定矩阵, 我们可以使用 Cholesky 分解, 即 $A = L^T L$, 式中 L 是下三角阵, 每行是一个 v_i 。

完整的 Max Cut 算法如下:

1. 通过上述的转化, 将目标函数转化为 (4) 的形式, 此时问题是一个 SDP 问题。求解得到 $\mathbb{U} = \{v_1, \dots, v_n\} \subseteq \mathbb{S}^n$
2. 在 $\mathbb{S}^n$ 上随机选取一个向量 r , 并根据 r 将 $\mathbb{U}$ 分类:

超平面

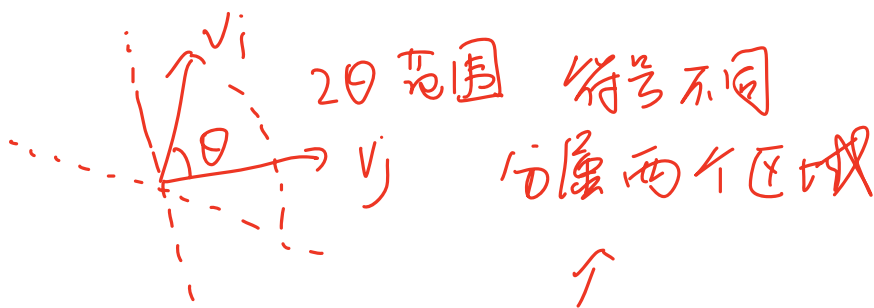
$$S = \{i \mid \langle v_i, r \rangle \geq 0\}$$

$$V \setminus S = \{i \mid \langle v_i, r \rangle < 0\}$$

3. $\{S, V \setminus S\}$ 是一个 Max Cut 的解。

3 算法分析

我们尝试分析算法的近似效果。



$$\mathbb{E}[w(S, V \setminus S)] = \sum_{i < j} w_{ij} \cdot \text{Prob}[\text{sgn}(\langle v_i, r \rangle) \neq \text{sgn}(\langle v_j, r \rangle)] \quad (8)$$

$$= \sum_{i < j} w_{ij} \frac{2 \arccos(\langle v_i, v_j \rangle)}{2\pi} \quad \text{来角} \theta \quad (9)$$

$$= \frac{1}{\pi} \sum_{i < j} w_{ij} \arccos(\langle v_i, v_j \rangle) = \Delta \quad (10)$$

Remark 3.1. 请注意, $\{v_1, v_2, \dots, v_n\}$ 仍然是在 $\mathbb{S}^n$, 而不是 $\mathbb{S}^2$ 上, 但考虑到两两向量之间时, 其组成的空间是 2 维的。

再考虑最优值 OPT 的一个上界 U, 得到我们结果的近似比 $\alpha = \frac{\Delta}{\text{OPT}} \geq \frac{\Delta}{U}$ 。由于问题 B 为 A 放松得到的, 也就是说问题 B 的最优解一定大于 A 的最优解。

问题 B 的目标函数是 $\frac{1}{2} \sum_{i < j} w_{ij} (1 - \langle v_i, v_j \rangle)$, 那么我们可以推出近似比。

$\Delta > 0.878 \text{ OPT}$

得到的解大于
0.878 倍最优解

$$\alpha = \frac{2 \sum_{i < j} w_{ij} \arccos(\langle v_i, v_j \rangle)}{\pi \sum_{i < j} w_{ij} (1 - \langle v_i, v_j \rangle)} \quad (11)$$

$$\geq \frac{2}{\pi} \min \frac{\arccos(\langle v_i, v_j \rangle)}{(1 - \langle v_i, v_j \rangle)} \quad \text{大于最小的} \quad (12)$$

$$\geq \frac{2}{\pi} \min \frac{\theta}{1 - \cos \theta} \approx 0.878 \quad (13)$$

根据之前的 Markov 不等式, 我们还能得到一些结果。

$1 - \alpha$

$$\mathbb{E}\left[\frac{\text{OPT} - \Delta}{\text{OPT}}\right] < 0.122 \Rightarrow \quad (14)$$

$$\text{Prob}\left[\frac{\text{OPT} - \Delta}{\text{OPT}} > 0.122(1 + \epsilon)\right] < \frac{\mathbb{E}\left[\frac{\text{OPT} - \Delta}{\text{OPT}}\right]}{0.122(1 + \epsilon)} \quad (15)$$

$$< \frac{1}{1 + \epsilon} \quad (16)$$

$$(17)$$

通过 Lecture1 中的 Union Bound, 可以通过多次重复, 提高概率, 这也是随机算法中常见的操作。

取 $\epsilon = 1$

4

$$\text{Prob}\left[\frac{\text{OPT} - \Delta}{\text{OPT}} > 0.244\right] < \frac{1}{2}$$

$$\text{Prob}[\Delta > 0.756 \text{ OPT}] > \frac{1}{2}$$

4 补充

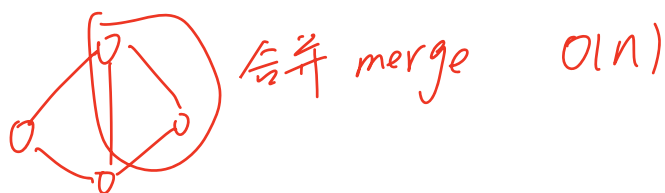
- Max Cut 问题是 Apx-Hard 问题，不存在现有的 PTAS 算法。且对于近似比 >0.941 的情况均为 NPH 问题
- 如果 Unique Games Conjecture 正确，那么 0.878 已经是最好的近似比。UG 猜想与本章的关系在于，如果其成立，那么 SDP 可以为一大类的近似问题提供**最优**的近似比，其中也包含 Max Cut。其提出者为 Subhash Khot[1]。可以参考的一些 note: [stanford Khot](#)

References

- [1] S. Khot. On the power of unique 2-prover 1-round games. In *Proceedings of the thirty-fourth annual ACM symposium on Theory of computing*, pages 767–775, 2002.

最小割 min-cut

① contract



② Alg

while $|V| > 2$

随机选一条边 $e \sim \frac{w_e}{\sum w_f}$

Contract e

边数 $m = O(n^2)$

Time = $O(n^2)$

Return 最终两部分 $S, V \setminus S$

③ 最优 min-cut 为 K $\bigcirc \equiv \bigcirc$ $\deg(v) \geq k$

$$W(E) \geq n \cdot \frac{k}{2} = \frac{1}{2}nk$$

随机抽选一条边 contract

$$P[e \in \text{option mincut}] = \frac{K}{\frac{1}{2}nk} = \frac{2}{n}$$

$$P[e \& \dots] = 1 - \frac{2}{n}$$

总体成功概率

$$(1 - \frac{2}{n})(1 - \frac{2}{n-1}) \dots (1 - \frac{2}{3})$$

$$= \frac{n-2}{n} \cdot \frac{n-3}{n-1} \dots \frac{2}{4} \cdot \frac{1}{3}$$

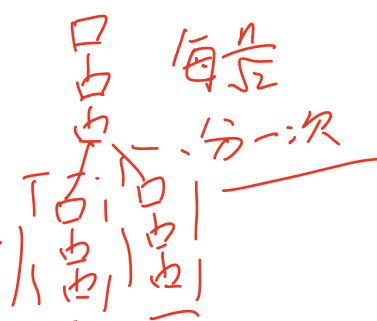
$$= \frac{2}{n(n-1)} = O\left(\frac{1}{n^2}\right) \quad O(n^2) \quad \underline{O(n^4)}$$

Alg 2 Recursive cut

$(n^2 \geq 2)$

if $n=2$

return the mincut

else $G_L \leftarrow G \quad G_R \leftarrow G$ (两个 copy) 

(1) contract G_L down to $\lceil \frac{n}{\sqrt{2}} \rceil$ vertices

$C_L = \text{Recursive cut}(G_L)$ // 递归

(2) contract G_R down to $\lceil \frac{n}{\sqrt{2}} \rceil$ vertices

$C_R = \text{Recursive cut}(G_R)$

(3) return $\min\{C_L, C_R\}$

$$T(n) = O(n^2) + 2T\left(\frac{n}{\sqrt{2}}\right)$$

$$\underline{T(n) = n^2(\log n)}$$

成功概率: $P_n = 1 - \left(1 - \frac{1}{n}\right)^2$

↑
主问题成功概率

四项 P 成功 q 不成功 $\underline{p^2 + 2pq} = \underline{(1-q^2)} \dots$

$$P(n) \geq (1 - (1 - P(n/\sqrt{2}))^2) / 2$$

$$\approx O\left(\frac{1}{\log n}\right)$$

整体 $n^2 \log n \cdot \log n = O(n^2 \log^2 n)$